# Sri Lanka Institute of Information Technology




## SOFTWARE REQUIREMENTS SPECIFICATION
## Check List 01

| | |
|---|---|
| **Project ID** | R26-CS-011 |
| **Project Title** | A Lightweight Detect, Defense, and User Awareness System for AI-Driven Scams |
| **Student Details** | |
| **Names** | **Student ID** |
| **IT22277268** | Aberathna H.M.M.K |
| **IT22299048** | Vidara S V |
| **IT22628718** | Priyadewa P D J C |
| **Supervisor** | Mr. Dinith Primal |
| **Date of Submission** | 18 May 2026 |

**B.Sc. (Hons) in Information Technology**

**Specializing in Cyber Security**

# Table of Contents


1 Introduction ..... 4

1.1 Purpose ..... 4

1.2 Scope ..... 4

1.3 Definitions, Acronyms, and Abbreviations ..... 4

1.5 Overview ..... 6

2 Overall Description ..... 7

2.1 Product Perspective ..... 7

2.1.1 System Interfaces ..... 7

2.1.2 User Interfaces ..... 8

2.1.3 Hardware Interfaces ..... 8

2.1.4 Software Interfaces ..... 8

2.1.5 Communication Interfaces ..... 9

2.1.6 Memory Constraints ..... 9

2.1.7 Operations ..... 9

2.1.8 Site Adaptation Requirements ..... 9

2.2 Product Functions ..... 10

2.3 User Characteristics ..... 10

2.4 Constraints ..... 11

2.5 Assumptions and Dependencies ..... 11

2.6 Apportioning of Requirements ..... 12

3 Specific Requirements ..... 13

3.1 External Interface Requirements ..... 13

3.1.1 User Interfaces ..... 13

3.1.2 Hardware Interfaces ..... 14

3.1.3 Software Interfaces ..... 14

3.1.4 Communication Interfaces ..... 14

3.2 Classes / Objects ..... 15

3.2.1 Email Message ..... 15

3.2.2 PreFilterEngine ..... 15

3.2.3 ScamClassifier ..... 16

3.2.4 AIGenerationDetector ..... 16

3.2.5 RiskScorer ..... 17

3.2.6 AwarenessEngine ..... 18

3.2.7 ResponseController ..... 18

3.2.8 VulnerabilityProfiler ..... 19
3.2.9 SteganographyDetector ..... 20
3.2.10 SHAPExplainer ..... 20
3.2.11 DetectionLogger ..... 21
3.2.12 WebDashboard ..... 21
3.3 Performance Requirements ..... 22
3.4 Design Constraints ..... 22
3.5 Software System Attributes ..... 23
3.5.1 Reliability ..... 23
3.5.2 Availability ..... 23
3.5.3 Security ..... 23
3.5.4 Maintainability ..... 24
3.6 Other Requirements ..... 24
4 Supporting Information ..... 25
4.1 Appendices ..... 25
Appendix A — System Data Flow Summary ..... 25
Appendix B — Risk Level Response Summary ..... 26
Appendix C — Technology Stack Summary ..... 26
Appendix D — System Hardware Requirements ..... 26
1.4 References ..... 28

# 1 Introduction

## 1.1 Purpose

This document constitutes the Software Requirements Specification (SRS) for the project titled "A Lightweight Detect, Defense, and User Awareness System for AI-Driven Scams" (Project ID: R26-CS-011). It defines the complete set of functional and non-functional requirements that the system must satisfy, establishes the scope and boundaries of development, and serves as the authoritative technical reference for the development team, academic supervisors, and evaluators throughout the project lifecycle.

This SRS covers all three integrated components of the system: the Lightweight Detection Model (Component 1), the Human-Centered Awareness and Defense System (Component 2), and the Explainable AI Steganographic Image Detection Module (Component 3). It further specifies the requirements of the Thunderbird extension, the local Python backend, and the React-based web dashboard that constitute the overall deployment architecture.

## 1.2 Scope

The scope of this SRS is bounded to the software aspects of the AI-Driven Scam Detection and Awareness System, specifically:

- The Thunderbird Web Extension that intercepts incoming emails and presents risk warnings to the user.
- The local Python backend hosts all machine learning models, detection pipelines, and REST API endpoints.
- The React-based web dashboard served at localhost, providing user awareness and analyst oversight functionality.
- The three detection and response components: AI-scam classification, human-centered awareness engine, and steganographic image analysis.
- The locally stored SQLite user vulnerability profile and detection log database.

The system is scoped to the Thunderbird email client (version 115 and above) running on Windows 10+, macOS 11+, or Ubuntu 20.04+. It operates exclusively on a single local machine with no cloud dependency. Email content does not leave the user's machine at any stage of processing.

Out of scope: integration with other email clients (e.g., Gmail, Outlook), cloud-based deployment, multilingual detection beyond English, and steganography detection in non-image file formats.

## 1.3 Definitions, Acronyms, and Abbreviations

| Term / Acronym | Definition |
|---|---|
| SRS | Software Requirements Specification |
| AI | Artificial Intelligence |
| ML | Machine Learning |
| LLM | Large Language Model – a generative AI model capable of producing human-like text |

| | |
|---|---|
| NLP | Natural Language Processing |
| DistilBERT | Distilled Bidirectional Encoder Representations from Transformers – a lightweight NLP model |
| CNN | Convolutional Neural Network |
| SHAP | SHapley Additive exPlanations – a framework for explaining ML model predictions |
| REST | Representational State Transfer – an architectural style for HTTP-based APIs |
| API | Application Programming Interface |
| SPF | Sender Policy Framework – email authentication protocol |
| DKIM | DomainKeys Identified Mail – email authentication protocol |
| DMARC | Domain-based Message Authentication, Reporting, and Conformance |
| LSB | Least Significant Bit – a steganographic encoding technique |
| DCT | Discrete Cosine Transform – used in JPEG compression and steganography |
| SRNet | Spatial-channel Residual Network – a CNN architecture for steganalysis |
| SQLite | Lightweight local relational database engine |
| SQLCipher | Open-source SQLite extension providing transparent AES-256 encryption |
| FastAPI | Modern, high-performance Python web framework for building REST APIs |
| PyTorch | An open-source ML framework used for training and inference of neural networks |
| spaCy | Industrial-strength NLP library in Python |
| Web Extension API | Browser extension API standard supported by Thunderbird (v115+) |
| localhost | Network address (127.0.0.1) refers to the local machine only |
| Risk Level | Categorical output of Component 1: Low (1–39), Medium (40–69), or High (70–100) |
| Steganography | The practice of concealing data within an ordinary-looking file, such as an image |
| Steganalysis | The detection and analysis of steganographic content in files |
| Vulnerability Profile | A persistent per-user record of scam exposure history and behavioral patterns |

## 1.5 Overview

The system addresses three interconnected problems in modern email security: the rise of AI-generated phishing emails that bypass traditional filters, the lack of contextual user education at the moment of threat, and the presence of steganographic payloads hidden in email image attachments that go entirely undetected by standard tools.

The system integrates directly into the Mozilla Thunderbird email client as a Web Extension. When an email is received, the extension forwards its content to a locally running Python backend. The backend applies a three-stage detection pipeline: a rule-based pre-filter, a DistilBERT-based scam classifier, and an AI-generation detector that produces a risk score and risk level (Low, Medium, or High). A human-centered awareness engine then generates personalized educational content tailored to the specific scam type and psychological manipulation technique detected. If the email contains image attachments, a CNN-based steganographic detector analyzes them and produces SHAP heatmaps identifying suspicious pixel regions.

All processing occurs locally. A React-based web dashboard, served at localhost:5000, provides users with an awareness feed and training interface, and provides analysts with a live security operations view. The system's primary users are individual email users seeking privacy-preserving, real-time protection against AI-driven scams.

# 2 Overall Description

## 2.1 Product Perspective

This system is a novel, integrated email security solution that combines real-time AI-driven scam detection, adaptive user awareness, and explainable steganographic image analysis within a single, locally deployed application. It is designed to operate as a standalone product embedded within the Thunderbird email client and does not depend on any external cloud service or third-party security platform.

The following table compares the system against major existing solutions to provide perspective on the product's positioning:

| Feature | This System | Spam Assassin | MS Defender (O365) | Gmail Filters | Mail Scanner |
|---|---|---|---|---|---|
| AI-generated scam detection | Yes (DistilBERT + stylometric) | No | Partial | Partial | No |
| Steganographic image analysis | Yes (SRNet/Ye-Net + SHAP) | No | No | No | No |
| User awareness & education | Yes (dynamic, personalized) | No | Limited | No | No |
| Explainable AI output | Yes (SHAP heatmaps) | No | No | No | No |
| User vulnerability profiling | Yes (adaptive, local) | No | No | No | No |
| Fully local / privacy-preserving | Yes | Yes | No (cloud) | No (cloud) | Yes |
| Thunderbird integration | Yes (native) | Partial | No | No | Partial |
| Risk scoring (0–100) | Yes | Yes (score-based) | No | No | No |

### 2.1.1 System Interfaces

The system interfaces with the host operating system at the following levels:

- Process management: The Python backend is launched as a background process via a native messaging host script when Thunderbird starts and terminates when Thunderbird exits.
- File system: The backend reads and writes to the local application data directory for the SQLite database, model files, and detection logs.

- Network (loopback only): All inter-component communication uses HTTP REST over the loopback interface (127.0.0.1). No external network access is required at runtime.
- Thunderbird Web Extension API: The extension interacts with Thunderbird via the message Displayed event and the message Display Scripts API.

### 2.1.2 User Interfaces

The system exposes two primary user interface surfaces:

- Thunderbird Inline Warning Panel: An inline notification banner rendered within the Thunderbird email view. Displays risk level indicator, key warning message, and a link to the full dashboard report. Appearance adapts based on risk level (soft banner for Low, prominent modal-style for Medium, blocking alert for High).
- Web Dashboard (localhost:5000): A React-based single-page application providing two distinct views — a User View for awareness and training, and an Analyst View for security operations and oversight. Accessible via a button in the Thunderbird extension.

### 2.1.3 Hardware Interfaces

The system has no direct hardware interface requirements beyond the host computer. Minimum hardware specifications are:

- RAM: 8 GB minimum (16 GB recommended for smooth ML model inference)
- Storage: Approximately 2 GB for model files, application data, and logs
- Processor: Any modern x86-64 CPU (GPU not required; inference targets CPU only)

### 2.1.4 Software Interfaces

| External Software | Interface Type | Purpose |
|---|---|---|
| Mozilla Thunderbird v115+ | Web Extension API | Email interception, inline warning display |
| Python 3.9+ | Runtime environment | Backend server and ML model execution |
| PyTorch | Python library | Neural network inference for Components 1 and 3 |
| Hugging Face Transformers | Python library | DistilBERT model loading and tokenization |
| spaCy | Python library | NLP processing in the Awareness Engine (Component 2) |
| SHAP | Python library | Explainability heatmap generation in Component 3 |
| SQLite + SQLCipher | Embedded database | Encrypted local storage of user profile and logs |
| React.js | Frontend framework | Web dashboard rendering |
| Node.js / npm | Build toolchain | Thunderbird extension build |

### 2.1.5 Communication Interfaces

- REST over HTTP (localhost only): The Thunderbird extension communicates with the Python backend via HTTP POST/GET requests to 127.0.0.1. Three primary endpoints exist: /classify (email content → risk assessment), /analyze-image (image data → steganography result), and /dashboard (aggregated data for the web dashboard).
- No external network communication: The system does not make any outbound network calls during normal operation. Internet access is only required during initial installation for downloading Python packages and ML model weights.

### 2.1.6 Memory Constraints

- The DistilBERT inference model requires approximately 250–300 MB of RAM when loaded.
- The SRNet/Ye-Net steganographic model requires approximately 150–200 MB of RAM when loaded.
- The total application memory footprint (backend + models + dashboard server) is expected not to exceed 1.5 GB under normal operation on a system with 8 GB RAM.
- The SQLite database is expected to grow at approximately 5–10 MB per month per user under typical usage.

### 2.1.7 Operations

Normal operations performed by the user:

- Receive and read emails in Thunderbird (detection occurs automatically).
- View inline risk warning banners within Thunderbird.
- Interact with or dismiss warnings based on risk level.
- Access the web dashboard to view awareness content, training sessions, and detection history.
- Override a quarantined or blocked email from the analyst dashboard view.

Special/administrative operations:

- Start/stop the backend server (handled automatically via the native messaging host).
- Export incident reports from the analyst dashboard.
- Review SHAP heatmaps for steganographically flagged image attachments.

### 2.1.8 Site Adaptation Requirements

- The system is designed for single-user local deployment. No server-side configuration or network infrastructure is required.
- The Python backend must be initialized once per machine to create the local database and download model weights (requires internet access at this step only).
- The Thunderbird extension must be installed as a temporary or signed extension via Thunderbird's add-on manager.
- The system supports Windows 10+, macOS 11+, and Ubuntu 20.04+ without platform-specific adaptation beyond standard Python and npm installation.

## 2.2 Product Functions

The major functions of the system are summarized below (detailed requirements are specified in Section 3):

- Real-time Email Interception: Automatically intercepts every incoming email in Thunderbird and submits it for analysis without user action.
- AI-Scam Classification: Classifies each email as scam or legitimate using a fine-tuned DistilBERT model, with a confidence score.
- AI-Generation Detection: Determines whether a classified scam email was generated by an LLM using stylometric analysis.
- Risk Scoring: Produces a numerical risk score (0–100) and categorical risk level (Low / Medium / High) for each email.
- Risk-Based Response Actions: Applies proportionate defensive actions — soft warning, quarantine, or auto-block — based on the risk level.
- Dynamic Awareness Content Generation: Generates personalized, contextual educational content for each detected scam, identifying the scam type and psychological manipulation technique.
- User Vulnerability Profiling: Maintains a persistent, adaptive profile per user tracking scam exposure history, response behavior, and susceptibility patterns.
- Adaptive Training Sessions: Generates interactive training exercises personalized to the user's vulnerability profile.
- Steganographic Image Detection: Analyzes image attachments in scam emails for hidden data using a CNN-based steganalysis model.
- SHAP Explainability: Produces pixel-level heatmaps identifying suspicious regions in flagged images, displayed on the analyst dashboard.
- Web Dashboard — User View: Provides inbox summary, awareness feed, training sessions, risk history, and profile insights.
- Web Dashboard — Analyst View: Provides live email queue, SHAP heatmaps, trend charts, vulnerability reports, manual override controls, and export tools.

## 2.3 User Characteristics

The system serves two distinct user categories:

| User Category | Characteristics | Technical Expertise |
|---|---|---|
| Individual Email User | Primary end-user of the system. Uses Thunderbird for personal or professional email. May have no cybersecurity background. Relies on the system for protection and education without needing to understand underlying ML models. | Low to moderate computer literacy. No cybersecurity expertise assumed. |
| Cybersecurity Analyst | Secondary user who accesses the analyst dashboard view. Reviews detection results, SHAP heatmaps, and trend data. May override quarantine decisions. Likely to be technically proficient. | High computer literacy. Moderate to high cybersecurity knowledge. |

## 2.4 Constraints

- Language Constraint: The DistilBERT model and awareness engine are trained and optimized for English-language emails only. Performance on non-English content is not guaranteed.
- Platform Constraint: The system is exclusively integrated with Mozilla Thunderbird (v115+). Other email clients are not supported.
- Local Processing Constraint: All computation occurs on the user's local machine. The system cannot leverage cloud-based processing or centralized threat intelligence feeds.
- Image Format Constraint: Steganographic analysis (Component 3) is limited to JPEG, PNG, BMP, and TIFF image formats. Other attachment types are not analyzed.
- Model Retraining Constraint: The ML models do not automatically retrain or update in response to new scam patterns. Model updates require a manual update process.
- Hardware Constraint: The system requires a minimum of 8 GB RAM. Performance may degrade on lower-specification machines.
- Network Constraint: All REST communication is restricted to the loopback interface. The system will not function in environments where localhost communication is blocked.
- Thunderbird Version Constraint: The Web Extension API features used (specifically message Displayed event and messageDisplayScripts) require Thunderbird version 115 (Supernova) or later.

## 2.5 Assumptions and Dependencies

- Mozilla Thunderbird (version 115 or later) is already installed and configured on the user's machine before system installation.
- Python 3.9 or later is available on the user's system, with the ability to install packages via pip.
- Node.js and npm are available on the user's system for building the Thunderbird extension.
- The user's machine meets the minimum hardware requirements: 8 GB RAM, 2 GB available storage, and a modern x86-64 CPU.
- Internet access is available during the initial installation phase for downloading Python package dependencies and ML model weights from Hugging Face. Internet access is not required during normal runtime operation.
- The user's email account is configured in Thunderbird and receives emails in MIME/multipart format (plain text, HTML, or mixed with attachments).
- Emails are assumed to be primarily in English. The system's detection accuracy is not guaranteed for non-English content.
- Image attachments, when present, are in one of the supported formats: JPEG, PNG.
- The user's operating system does not block inter-process communication over the loopback interface (127.0.0.1).
- The local machine has sufficient available storage (at least 2 GB) to accommodate model files, the SQLite database, and application data.

## 2.6 Apportioning of Requirements

Requirements will be implemented in the following priority order, reflecting the system's dependency chain and research milestones:

| Priority | Component / Feature | Rationale |
|---|---|---|
| 1 – Essential | Component 1: Detection Pipeline (pre-filter, DistilBERT classifier, AI-generation detector, risk scorer) | Core of the system. All other components depend on their output. Must be functional before integration. |
| 2 – Essential | Thunderbird Extension + Local Backend REST API Integration | Required to feed real emails into Component 1 and return results to the user interface. |
| 3 – Essential | Component 2: Awareness Engine and Risk-Based Response Actions | Directly depends on Component 1's risk output. Provides the user-facing defense capability. |
| 4 – Desirable | Web Dashboard (User View and Analyst View) | Depends on Components 1 and 2. Provides full visibility and oversight, but the core protection function operates without it. |
| 5 – Desirable | Component 2: User Vulnerability Profiling and Adaptive Training | Component 2 extends with long-term resilience building. Requires an accumulated detection history. |
| 6 – Optional | Component 3: Steganographic Image Detection and SHAP Explainability | Most technically complex. Does not affect core email protection. Added after Components 1 and 2 are validated. |

# 3 Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

UI-01: Thunderbird Inline Warning Banner

- The extension shall render an inline warning panel within the Thunderbird email reading panel upon receiving a risk assessment from the backend.
- The panel shall display risk level label (Low / Medium / High), a color-coded indicator (green/amber/red), a plain-language warning message, and a link to the full dashboard report.
- For Low risk: the panel shall appear as a non-blocking banner at the top of the email view. The user may dismiss it or continue reading the email.
- For Medium risk: the panel shall appear as a prominent warning requiring user acknowledgement (click to confirm) before the email body is displayed.
- For High risk: the panel shall block the email body entirely and display a detailed alert message. The email shall not be accessible without an explicit override action from the analyst dashboard.

UI-02: Web Dashboard — User View (localhost:5000)

- Inbox Summary: Displays a list of recently received emails with their assigned risk level indicators.
- Awareness Feed: For each detected scam, it displays the scam type classification, detected psychological trigger, extracted high-risk keywords, and a plain-language explanation with best practice guidance.
- Training Sessions: Presents interactive, adaptive training exercises generated from the user's vulnerability profile.
- Risk History: A timeline visualization of scam emails received over configurable time ranges.
- Profile Insights: Summary view of the user's most encountered scam types and psychological manipulation tactics.

UI-03: Web Dashboard — Analyst View (localhost:5000)

- Live Email Queue: Tabular display of processed emails with risk scores, classification labels, and timestamps.
- Steganography Results: Side-by-side display of original attachment image and SHAP heatmap, with detection confidence and technique identification.
- Trend Charts: Line and bar charts showing detection frequency, scam type distribution, and risk level distribution over time.
- User Vulnerability Report: Aggregated profile data and behavioral analysis.
- Manual Override Controls: Ability to release quarantined or blocked emails after analyst review.
- Export Tools: Export incident reports and forensic documentation in a standard format (PDF or CSV).

### 3.1.2 Hardware Interfaces

The system has no direct hardware interface requirements. It operates entirely in software on a standard x86-64 personal computer. The minimum hardware specification is: 8 GB RAM, 2 GB storage, and a modern multi-core CPU. No specialized hardware (GPU, HSM, hardware token) is required.

### 3.1.3 Software Interfaces

SI-01: Thunderbird Web Extension API

- The extension shall use the messages-on-Displayed event to intercept emails as they are opened.
- The extension should use the messageDisplayScripts API to inject the inline warning panel into the email reading pane.

SI-02: Python Backend (FastAPI or Flask)

- The backend shall expose a POST /classify endpoint accepting email content (subject, body, headers, sender metadata) in JSON format and returning a risk score, risk level, scam type, AI-generation flag, and triggered keywords.
- The backend shall expose a POST /analyze-image endpoint accepting base64-encoded image data and returning detection result, confidence score, technique classification, and SHAP heatmap data.
- The backend shall expose a GET /dashboard endpoint returning aggregated detection history, trend data, and user profile data for the web dashboard.

SI-03: Hugging Face Transformers / PyTorch

- The backend shall load the fine-tuned DistilBERT model from a local model directory at startup.
- The backend shall load the SRNet or Ye-Net CNN model from a local model directory at startup.

### 3.1.4 Communication Interfaces

- All communication between the Thunderbird extension and the local backend shall use HTTP/1.1 REST over the loopback interface (127.0.0.1).
- All requests and responses shall use the JSON content type (application/Json).
- The backend server shall listen on a fixed local port (default: 5000). The port shall be configurable.
- The system shall not make any outbound connections to external servers during normal operation.
- All communication shall remain within the local machine boundary at all times.

## 3.2 Classes / Objects

This section specifies the detailed functional requirements organized by domain classes. Each requirement is marked as Essential (E), Desirable (D), or Optional (O).

### 3.2.1 Email Message

Represents an email intercepted by the Thunderbird extension and submitted to the backend for analysis.

| Requirement ID | Description | Priority |
|---|---|---|
| EM-01 | The system shall extract the subject line, body text (plain text and HTML), sender address, sender display name, Reply-To address, and all email headers from every intercepted email. | E |
| EM-02 | The system shall extract all image attachments (JPEG, PNG, BMP, TIFF) from intercepted emails and make them available to Component 3. | E |
| EM-03 | The system should operate in read-only mode on email content. It shall not modify, delete, or reclassify emails within Thunderbird's internal storage. | E |
| EM-04 | The system shall process multipart MIME emails including plain text, HTML, and mixed content types. | E |
| EM-05 | The system shall handle emails with no attachments, no body text, or missing headers without error. | E |
| EM-06 | The system shall truncate email body text to a maximum of 512 tokens before submission to the DistilBERT model, consistent with the model's maximum context length. | E |

### 3.2.2 PreFilterEngine

Performs rapid rule-based analysis of email metadata and headers before invoking any ML model.

| Requirement ID | Description | Priority |
|---|---|---|
| PF-01 | The PreFilterEngine shall validate SPF, DKIM, and DMARC authentication records for each incoming email. | E |
| PF-02 | The PreFilterEngine shall assess the sender domain age and assign a reputation score based on known domain age heuristics. | E |

| | | |
|---|---|---|
| PF-03 | The PreFilterEngine shall perform subject line pattern matching against a maintained library of known scam subject templates. | E |
| PF-04 | The PreFilterEngine shall extract all URLs from the email body and check them against a local list of known malicious domains. | E |
| PF-05 | The PreFilterEngine shall complete its analysis in under 50ms per email. | E |
| PF-06 | The PreFilterEngine shall pass its output (authentication flags, reputation scores, URL match results) to the ScamClassifier as supplementary features. | E |

### 3.2.3 ScamClassifier

Fine-tuned DistilBERT model that classifies emails as scam or legitimate.

| Requirement ID | Description | Priority |
|---|---|---|
| SC-01 | The ScamClassifier shall use a fine-tuned DistilBERT-base-uncased model as its primary classification engine. | E |
| SC-02 | The ScamClassifier shall accept tokenized email subject and body text (combined, max 512 tokens) as input. | E |
| SC-03 | The ScamClassifier shall output a binary classification label (scam / legitimate) and a floating-point confidence score (0.0–1.0). | E |
| SC-04 | The ScamClassifier shall achieve an inference time of under 200ms per email on a CPU-only environment. | E |
| SC-05 | Emails classified as legitimate shall not be forwarded to the AIGenerationDetector. | E |
| SC-06 | The ScamClassifier model weights shall be stored locally and loaded at backend startup. | E |

### 3.2.4 AIGenerationDetector

The secondary classifier determines whether a scam email was generated by a large language model.

| Requirement ID | Description | Priority |
|---|---|---|
| AG-01 | The AIGenerationDetector shall only be invoked for emails classified as scam by the ScamClassifier. | E |

| | | |
|---|---|---|
| AG-02 | The AIGenerationDetector shall analyze stylometric features, including sentence-level perplexity, burstiness, lexical diversity, and semantic coherence. | E |
| AG-03 | The AIGenerationDetector shall output a binary classification label (AI-generated / human-written) and a confidence score. | E |
| AG-04 | The AI-generation classification result shall be included in the final risk assessment output passed to the RiskScorer and AwarenessEngine. | E |
| AG-05 | The AIGenerationDetector shall complete inference in under 100ms per email on CPU. | D |

### 3.2.5 RiskScorer

Produces the final numerical risk score and categorical risk level for each email.

| Requirement ID | Description | Priority |
|---|---|---|
| RS-01 | The RiskScorer shall combine outputs from the PreFilterEngine, ScamClassifier, and AIGenerationDetector into a single risk score in the range 0–100. | E |
| RS-02 | The RiskScorer shall assign categorical risk levels: Low (1–39), Medium (40–69), High (70–100). | E |
| RS-03 | The presence of image attachments in a scam-classified email shall contribute positively to the risk score. | E |
| RS-04 | Confirmed AI-generation shall increase the risk score contribution from the AIGenerationDetector component. | E |
| RS-05 | The RiskScorer output shall be forwarded to the AwarenessEngine and logged to the database. | E |

### 3.2.6 AwarenessEngine

Generates personalized, contextual educational content for every detected scam.

| Requirement ID | Description | Priority |
|---|---|---|
| AE-01 | The AwarenessEngine shall classify each detected scam into one of the following types: phishing, spear-phishing, romance scam, investment fraud, or other. | E |
| AE-02 | The AwarenessEngine shall identify the primary psychological manipulation technique used in the email from the following set: urgency, authority impersonation, fear, reciprocity, social proof, or scarcity. | E |
| AE-03 | The AwarenessEngine shall extract and highlight high-risk keywords and phrases from the email body. | E |
| AE-04 | The AwarenessEngine shall generate a plain-language explanation of how the detected scam works and who it typically targets. | E |
| AE-05 | The AwarenessEngine shall generate specific best-practice actions the user should take in response to the detected scam. | E |
| AE-06 | All awareness content shall be generated and returned within 100ms of receiving the risk assessment. | D |
| AE-07 | The AwarenessEngine shall store each generated awareness event (scam type, technique, keywords) in the local database for profiling. | E |

### 3.2.7 ResponseController

Applies the appropriate defensive action based on the email's risk level.

| Requirement ID | Description | Priority |
|---|---|---|
| RC-01 | For Low risk emails, the ResponseController shall instruct the extension to display a soft, non-blocking inline warning banner. The email shall remain accessible. | E |
| RC-02 | For Medium risk emails, the ResponseController shall instruct the extension to quarantine the email and display a prominent warning requiring user acknowledgement. | E |

| | | |
|---|---|---|
| RC-03 | For High risk emails, the ResponseController shall instruct the extension to auto-block the email and move it to a dedicated quarantine folder. | E |
| RC-04 | High risk emails shall not be accessible to the user without an explicit override action performed from the analyst dashboard. | E |
| RC-05 | The ResponseController shall pass the risk level, scam type, and awareness content to the Thunderbird extension for inline display. | E |

### 3.2.8 VulnerabilityProfiler

Maintains and updates the persistent per-user vulnerability profile.

| Requirement ID | Description | Priority |
|---|---|---|
| VP-01 | The VulnerabilityProfiler shall maintain a persistent profile for each user in the local SQLite database. | E |
| VP-02 | The profile shall record the history of scam emails received, categorized by scam type and risk level. | E |
| VP-03 | The profile shall record the user's response to each warning: dismissed, interacted with, or reported. | E |
| VP-04 | The profile shall track which psychological trigger types the user has historically been most exposed to. | E |
| VP-05 | The profile shall track user engagement with awareness content and training sessions. | D |
| VP-06 | The VulnerabilityProfiler shall generate adaptive training sessions targeting the scam types and manipulation techniques most prevalent in the user's history. | D |
| VP-07 | All profile data shall be stored in the local SQLite database encrypted via SQLCipher (AES-256). | E |

### 3.2.9 SteganographyDetector

CNN-based component that detects hidden data in image attachments of scam emails.

| Requirement ID | Description | Priority |
|---|---|---|
| SD-01 | The SteganographyDetector shall only be invoked when Component 1 has classified the email as a scam, and the email contains at least one image attachment. | E |
| SD-02 | Supported image formats for analysis: JPEG, PNG, BMP, TIFF. | E |
| SD-03 | The SteganographyDetector shall use a CNN model (SRNet or Ye-Net architecture) for pixel-level steganographic analysis. | E |
| SD-04 | The SteganographyDetector shall output a binary detection result (steganographic content present / not detected) and a confidence score (0–100%). | E |
| SD-05 | If steganographic content is detected, a secondary classifier shall identify the technique: LSB substitution, DCT-domain embedding, palette-based steganography, or WOW/HUGO adaptive steganography. | D |
| SD-06 | The SteganographyDetector shall complete analysis in under 2 seconds per image. | E |
| SD-07 | A positive steganography detection shall be factored into the overall email risk level. | E |

### 3.2.10 SHAPExplainer

Generates pixel-level explainability heatmaps for steganographic image detections.

| Requirement ID | Description | Priority |
|---|---|---|
| SE-01 | The SHAPExplainer shall use SHAP with a Gradient Explainer to generate pixel-level heatmaps for every image flagged by the SteganographyDetector. | E |
| SE-02 | The heatmap shall highlight the specific regions of the image identified as anomalous by the model. | E |
| SE-03 | The SHAPExplainer shall output the annotated heatmap image alongside the original image for display in the analyst dashboard. | E |

| | | |
|---|---|---|
| SE-04 | SHAP heatmap generation shall complete within the overall 2-second image analysis target (inclusive of SteganographyDetector inference). | D |
| SE-05 | The heatmap shall be displayed alongside detection confidence and technique classification in the analyst dashboard. | E |

### 3.2.11 DetectionLogger

All detection events, risk assessments, and awareness data persists to the local database.

| Requirement ID | Description | Priority |
|---|---|---|
| DL-01 | The DetectionLogger shall log every processed email with: timestamp, sender, subject (hashed for privacy if required), risk score, risk level, scam type, AI-generation flag, and triggered keywords. | E |
| DL-02 | The DetectionLogger shall log every steganographic analysis result with: timestamp, image filename, detection result, confidence score, and technique identified. | E |
| DL-03 | The DetectionLogger shall log all user interactions with warning panels (dismissed, interacted, reported). | E |
| DL-04 | All logged data shall be stored in the local SQLite database encrypted via SQLCipher. | E |
| DL-05 | The DetectionLogger shall expose aggregated log data to the /dashboard REST endpoint for consumption by the web dashboard. | E |

### 3.2.12 WebDashboard

The React-based local web application provides user and analyst interfaces.

| Requirement ID | Description | Priority |
|---|---|---|
| WD-01 | The dashboard shall be served exclusively at http://localhost:5000 and shall not be accessible from any external network address. | E |
| WD-02 | The dashboard shall provide a User View with: inbox summary, awareness feed, training sessions, risk history timeline, and profile insights. | E |

| | | |
|---|---|---|
| WD-03 | The dashboard shall provide an Analyst View with: live email queue, SHAP heatmap viewer, scam trend charts, user vulnerability report, manual override controls, and export tools. | E |
| WD-04 | The dashboard shall load its initial data within 500ms of page open. | E |
| WD-05 | The dashboard shall be accessible from a button within the Thunderbird extension panel. | E |
| WD-06 | The dashboard shall support export of incident reports in PDF or CSV format. | D |

## 3.3 Performance Requirements

| Operation | Target Response Time | Measurement Condition |
|---|---|---|
| Email pre-filter (rule-based) | < 50ms | Per email, CPU-only, standard consumer hardware |
| Scam classification — DistilBERT inference | < 200ms | Per email, CPU-only, 8 GB RAM, 512-token input |
| AI-generation detection | < 100ms | Per email, CPU-only, following scam classification |
| Awareness content generation | < 100ms | Per email, following risk scoring |
| End-to-end: email arrival to inline warning display | < 500ms | Text-only email, no image attachments |
| Steganographic image analysis (Component 3) | < 2 seconds | Per image, CPU-only, standard resolution |
| Dashboard data load | < 500ms | Full page load from localhost |
| Backend server startup | < 30 seconds | Including model loading from local disk |

## 3.4 Design Constraints

- Privacy by Design: No email content, metadata, or user data shall be transmitted to any server outside the user's local machine. This is an absolute constraint that cannot be relaxed in any deployment configuration.
- Local Deployment Only: The system shall not require any cloud infrastructure, external API keys, or remote service endpoints for normal operation.
- Read-Only Email Access: The system shall not modify, delete, move, or reclassify email messages within Thunderbird's internal mail storage, except for the quarantine action (moving

High-risk emails to a dedicated local quarantine folder) which is initiated as an explicit defensive response.

- Thunderbird Extension Standards: The extension shall comply with the Thunderbird WebExtension API specification and shall not use deprecated APIs or undocumented internal interfaces.
- CPU-Only Inference Constraint: The ML models (DistilBERT and SRNet/Ye-Net) shall be configured for CPU-only inference. GPU acceleration is optional and not required.
- Encrypted Storage: All user data, detection logs, and vulnerability profiles shall be stored in a SQLite database encrypted with SQLCipher. Plaintext storage of sensitive data is not permitted.
- Single-User Architecture: The system is designed for single-user local deployment. Multi-user, shared-machine, or networked deployment scenarios are outside the design scope.

## 3.5 Software System Attributes

### 3.5.1 Reliability

- The backend server shall handle malformed or unexpected email content (missing headers, empty body, unsupported encoding) gracefully without crashing. It shall return an appropriate error response to the extension.
- If the ML model inference fails for any email, the system shall log the failure and return a default Low risk assessment rather than blocking the user's email access.
- The system shall recover automatically from backend process crashes via the native messaging host watchdog mechanism.
- The SQLite database shall use WAL (Write-Ahead Logging) mode to prevent data corruption in the event of an unexpected shutdown.

### 3.5.2 Availability

- The backend server shall start automatically when Thunderbird is launched and terminate cleanly when Thunderbird exits.
- The system shall be available for email analysis within 30 seconds of Thunderbird startup (model loading time).
- In the event that the backend is unavailable (e.g., startup failure), the extension shall display a clear notification to the user and allow normal Thunderbird operation without interference.

### 3.5.3 Security

- All user vulnerability profile data and detection logs shall be encrypted at rest using SQLCipher with AES-256 encryption.
- The web dashboard shall only be accessible from the loopback interface (127.0.0.1). It shall not bind to any external network interface.
- The backend REST API shall only accept connections from localhost. Requests from external IP addresses shall be rejected.
- The system shall not store full email body text in the database. Only metadata, extracted keywords, and classification results shall be logged.
- The Thunderbird extension shall request only the minimum required permissions from the Thunderbird WebExtension API.

### 3.5.4 Maintainability

- The system shall be organized as three clearly separated components (Component 1, 2, 3) with well-defined REST API interfaces between them, enabling independent updating or replacement of individual components.
- The DistilBERT and CNN model weights shall be stored in a dedicated /models directory. Updating a model shall require only replacing the model file, without code changes.
- The rule-based pre-filter's scam subject template library and malicious domain list shall be stored as external configuration files, updatable without code changes.
- The codebase shall include inline documentation and a developer setup guide sufficient for a new contributor to configure and run the system from source.

## 3.6 Other Requirements

- The system's awareness content shall be written in plain, accessible English at no higher than a general adult reading level. Technical jargon shall not be used in user-facing explanations.
- The installer or setup script shall verify that Thunderbird version 115+ and Python 3.9+ are present on the system before proceeding with installation, and shall display a clear error message if requirements are not met.
- The system shall log all errors and exceptions from the backend to a local log file for debugging purposes.
- The system shall not display any telemetry consent dialogs, advertisements, or external account registration prompts.

# 4 Supporting Information

## 4.1 Appendices

### Appendix A — System Data Flow Summary

| Step | Action | Component |
|---|---|---|
| 1 | Email arrives in Thunderbird inbox | Thunderbird |
| 2 | Extension intercepts email via messageDisplayed event | Thunderbird Extension |
| 3 | Extension extracts subject, body, headers, and image attachments | Thunderbird Extension |
| 4 | Content is sent to backend via POST /classify (localhost) | REST API |
| 5 | PreFilterEngine performs metadata and header rule checks | Component 1 |
| 6 | ScamClassifier (DistilBERT) classifies email as scam or legitimate | Component 1 |
| 7 | If scam: AIGenerationDetector assesses LLM origin via stylometric analysis | Component 1 |
| 8 | RiskScorer combines all signals into score (0–100) and level (Low/Med/High) | Component 1 |
| 9 | If image attachments present: POST /analyze-image to SteganographyDetector | Component 3 |
| 10 | SHAPExplainer generates pixel heatmap for flagged images | Component 3 |
| 11 | AwarenessEngine generates scam type, psychological trigger, keywords, guidance | Component 2 |
| 12 | ResponseController determines defensive action (warn / quarantine / block) | Component 2 |
| 13 | Backend returns full assessment JSON to the extension | REST API |
| 14 | Extension renders inline warning panel in Thunderbird email view | Thunderbird Extension |
| 15 | All results are logged to encrypted SQLite database | DetectionLogger |
| 16 | Results visible in web dashboard at localhost:5000 | Web Dashboard |

## Appendix B — Risk Level Response Summary

| Risk Level | Score Range | Email Accessible? | User Action Required | Analyst Override Available? |
|---|---|---|---|---|
| Low | 1 – 39 | Yes (immediately) | None — may dismiss banner | No |
| Medium | 40 – 69 | Yes (after acknowledgement) | Must acknowledge warning before reading | No |
| High | 70 – 100 | No (blocked) | Cannot access without analyst override | Yes |

## Appendix C — Technology Stack Summary

| Layer / Component | Technology |
|---|---|
| Thunderbird Extension | JavaScript, WebExtension API (Thunderbird v115+) |
| Local Backend Server | Python 3.11, FastAPI or Flask |
| Component 1 — Scam Classifier | Hugging Face Transformers, DistilBERT-base-uncased, PyTorch |
| Component 1 — AI-Generation Detector | Custom stylometric classifier, PyTorch |
| Component 2 — Awareness Engine | Python, spaCy, custom rule engine |
| Component 2 — Vulnerability Profiler | Python, SQLite |
| Component 3 — Steganography Detector | PyTorch, SRNet or Ye-Net CNN |
| Component 3 — SHAP Explainer | SHAP library (Gradient Explainer) |
| Web Dashboard | React.js, served via local dev server or Flask static |
| Database | SQLite + SQLCipher (AES-256 encrypted) |
| API Communication | REST over HTTP, loopback interface only (127.0.0.1) |
| Build Toolchain | Python virtual environment (venv), Node.js, npm |

## Appendix D — System Hardware Requirements

| Requirement | Minimum | Recommended |
|---|---|---|
| Operating System | Windows 10, macOS 11, Ubuntu 20.04 | Windows 11, macOS 13, Ubuntu 22.04 |
| RAM | 8 GB | 16 GB |
| Storage | 2 GB available | 5 GB available |
| Processor | x86-64 dual-core CPU | x86-64 quad-core CPU |
| GPU | Not required | Not required |
| Thunderbird Version | 115 (Supernova) | Latest stable |

| | | |
|---|---|---|
| Python Version | 3.9 | 3.11 |
| Node.js Version | 16.x | 20.x LTS |

## 1.4 References


| # | Reference | Relevance |
|---|---|---|
| 1 | Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. | Architecture basis for Component 1 scam classifier. |
| 2 | Ye, J. et al. (2017). Deep Learning Hierarchical Representations for Image Steganalysis. IEEE TIFS. | Ye-Net architecture basis for Component 3 steganalysis model. |
| 3 | Zhang, R. et al. (2019). Depth-wise Separable Convolutions and Multi-level Pooling for an Efficient Spatial CNN-based Steganalysis. IEEE TIFS. (SRNet reference) | SRNet architecture basis for Component 3 steganalysis model. |
| 4 | Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017. | Basis for SHAP explainability framework used in Component 3. |
| 5 | Mozilla Thunderbird Web Extension API Documentation. https://webextension-api.thunderbird.net/ | Thunderbird extension integration specification. |
| 6 | Hugging Face Transformers Library Documentation. https://huggingface.co/docs/transformers/ | DistilBERT model loading and inference interface. |
| 7 | FastAPI Documentation. https://fastapi.tiangolo.com/ | Local backend REST API framework. |
| 8 | SHAP Python Library Documentation. https://shap.readthedocs.io/ | SHAP Gradient Explainer implementation reference. |
| 9 | SpamAssassin Project. https://spamassassin.apache.org/ | Competing system referenced in product perspective. |
| 10 | Microsoft Defender for Office 365. https://learn.microsoft.com/en-us/microsoft-365/security/office-365-security/ | Competing system referenced in product perspective. |